Implementation Plan: Cinematic Portfolio Website

Oct 4, 2026 · @Naren

Companion to the PRD. The PRD says *what* to build; this plan says *exactly how, in what order, with what commands*. Work top to bottom. Each sprint ends with a **Done when** check, so you never move on with something half-working.

0. Prerequisites (Day 0)

Install: Node.js 20+, pnpm (`npm i -g pnpm`), Git, VS Code.

Create free accounts: GitHub, Vercel, Neon (Postgres), Resend (email), Cloudflare (Turnstile + DNS), Sentry, Upstash (Redis).

Gather content before coding (this is the #1 cause of delays):

- Showreel link + 6 hero projects (video URL, thumbnail, brand, title, description, credits)
- 6 vertical reels (9:16)
- 15–20 behind-the-scenes photos
- Client logos (with permission), service descriptions, FAQs, team bios
- Domain name, brand colours, 2 fonts (one display, one body)

1. Folder Structure

```
portfolio/
├─ app/
│  ├─ (site)/
│  │  ├─ page.tsx                # Home
│  │  ├─ portfolio/page.tsx
│  │  ├─ portfolio/[slug]/page.tsx
│  │  ├─ reels/page.tsx
│  │  ├─ services/[slug]/page.tsx
│  │  ├─ clients/[slug]/page.tsx
│  │  ├─ blog/ and blog/[slug]/
│  │  ├─ about/ production-ecosystem/ vendor-registration/
│  │  └─ privacy-policy/ terms-of-service/
│  ├─ admin/                     # CMS (protected)
│  ├─ api/
│  │  ├─ leads/route.ts
│  │  ├─ vendors/route.ts
```

```
|  |  ├─ upload/route.ts
|  |  └─ revalidate/route.ts
|  ├─ layout.tsx  sitemap.ts  robots.ts  not-found.tsx
├─ components/
|  ├─ layout/ (Navbar, MobileMenu, Footer, RecDot, Cursor)
|  ├─ sections/ (Hero, Evidence, Architecture, OnSet, Feed, TrustedBy,
|  |             LibraryHorizontal, LibraryVertical, Services, Partner,
|  |             Brands, BlogTeaser, Faq, ContactBrief)
|  └─ ui/ (Button, VideoEmbed, Marquee, Accordion, CreditsSlate, Lightbox)
├─ lib/ (db.ts, email.ts, validators.ts, ratelimit.ts, auth.ts, seo.ts)
├─ prisma/ (schema.prisma, seed.ts)
├─ content/ (mock JSON used in Sprint 2)
├─ tests/ (unit + e2e)
└─ .env.example
```

2. Sprint 1: Foundation (Days 1–3)

Tasks

1. Create the GitHub repo, then run:

```
pnpm create next-app@latest portfolio --typescript --tailwind --eslint --app
--src-dir=false
cd portfolio
pnpm add gsap framer-motion lenis react-hook-form zod @hookform/resolvers
pnpm add prisma @prisma/client next-auth resend @vercel/blob
@upstash/ratelimit @upstash/redis
pnpm add -D prettier husky lint-staged vitest playwright
```

2. Create branches: `main` (production), `develop` (work). Protect `main`.
3. Add design tokens in `globals.css`:

```
:root { --bg:#050505; --fg:#f5f5f5; --muted:#8a8a8a; --accent:#ff2d2d; }
body { background:var(--bg); color:var(--fg); }
```

4. Load fonts with `next/font` (display + body).
5. Create `.env.example` listing every variable (see Section 9).
6. Connect the repo to Vercel now, so every push gets a preview URL from day one.

Done when: a blank branded page deploys to a Vercel preview URL.

3. Sprint 2: Global Layout and UI Kit (Days 4–7)

1. **Navbar** (sticky, hides on scroll down, shows on scroll up) with CTA button and phone link.
2. **MobileMenu**: full-screen overlay animated with Framer Motion; lock body scroll when open.
3. **RecDot**: CSS-only blinking dot.

```
@keyframes rec { 50% { opacity:.2 } }
.rec-dot { width:8px; height:8px; border-radius:50%; background:#ff2d2d;
animation:rec 1.2s infinite; }
```

4. **Lenis smooth scroll** provider in `layout.tsx`, disabled when `prefers-reduced-motion`.
5. **VideoEmbed** component: shows the thumbnail first, loads the iframe only on click or when scrolled into view.

```
function VideoEmbed({ id, poster }: { id: string; poster: string }) {
  const [on, setOn] = useState(false);
  return on ? (
    <iframe src={`https://www.youtube.com/embed/${id}?
autoplay=1&mute=1&playsinline=1&rel=0`}
      allow='autoplay; encrypted-media' className='w-full aspect-video' />
  ) : (
    <button onClick={() => setOn(true)} aria-label='Play video'>
      <img src={poster} alt='' className='w-full aspect-video object-cover'
/>
    </button>
  );
}
```

6. **Marquee** component (CSS keyframes, pause on hover), **Accordion**, **Button**, **Lightbox**, **CreditsSlate**.
7. Footer with links and socials.

Done when: layout renders on mobile and desktop, menu works, Lighthouse accessibility is 90+.

4. Sprint 3: Home Page Sections with Mock Data (Days 8–20)

Build in this order (easiest to hardest), each reading from `content/*.json`:

#	Section	Key implementation note
1	Hero	Background video (muted, loop, `playsInline`) + poster fallback; 2 buttons; showreel opens Lightbox
2	Architecture (3 phases)	Static layout; large numerals; links to `/services/[slug]`
3	Trusted By	Marquee of logos; `filter: grayscale(1)` → colour on hover/touch
4	On Set gallery	Duplicate the image list twice for seamless infinite loop; `next/image` lazy
5	Brands grid	Logo grid linking to `/clients/[slug]`
6	FAQ	Accessible accordion; tabs for groups
7	Contact brief	React Hook Form + Zod; chips; budget slider with tier labels (below)
8	Feed (swipe deck)	Horizontal scroll-snap on mobile; drag with Framer Motion on desktop
9	Library (horizontal + vertical)	Left list, right preview; active state in React state; credits slate below
10	Cinematic Evidence	GSAP ScrollTrigger pinning (below)
11	Partner + blog teaser	Static cards

Pinned scroll pattern (Cinematic Evidence):

```
gsap.registerPlugin(ScrollTrigger);
const slides = gsap.utils.toArray('.slide');
slides.forEach((slide, i) => {
  ScrollTrigger.create({
    trigger: slide, start: 'top top', end: '+=100%',
    pin: true, pinSpacing: false,
  });
});
```

Wrap in `gsap.context()` inside `useLayoutEffect` and call `ctx.revert()` on cleanup. Use `ScrollTrigger.matchMedia` to simplify the effect on small screens.

Budget slider tiers (example logic):

```
const tier = (v: number) =>
  v < 100000 ? 'Indie Production' :
  v < 500000 ? 'Commercial Grade' :
  v < 1500000 ? 'Premium Production' : 'Blockbuster Energy';
```

Done when: the full home page matches the Figma design on mobile and desktop, at 60fps scrolling on a mid-range phone.

5. Sprint 4: Inner Pages (Days 21–28)

1. `/portfolio`: grid + filters driven by URL query params (`?format=vertical&brand=...`).
2. `/portfolio/[slug]`: hero video, brief, credits, gallery, next/previous project. Use `generateStaticParams` and `generateMetadata`.
3. `/services/[slug]`, `/clients/[slug]`, `/about`, `/production-ecosystem`.
4. `/blog` + `/blog/[slug]` (MDX for now).
5. `/vendor-registration` form, legal pages, `not-found.tsx`.
6. SEO: `lib/seo.ts` helper, `sitemap.ts`, `robots.ts`, JSON-LD (Organization, VideoObject, Article, FAQPage).

Done when: every route in the PRD site map loads without errors using mock data.

6. Sprint 5: Database and APIs (Days 29–36)

1. Create the Neon database, copy `DATABASE_URL` to `.env`.
2. Core Prisma schema (`prisma/schema.prisma`):

```
generator client { provider = 'prisma-client-js' }
datasource db { provider = 'postgresql'  url = env('DATABASE_URL') }

enum Format { HORIZONTAL VERTICAL }
enum LeadStatus { NEW CONTACTED WON LOST }

model Client {
  id String @id @default(cuid())
  slug String @unique
  name String
  logoUrl String
  description String?
  order Int @default(0)
  published Boolean @default(true)
  projects Project[]
```

```
}

model Project {
  id String @id @default(cuid())
  slug String @unique
  title String
  category String
  format Format
  videoId String
  thumbnailUrl String
  description String
  credits Json
  year Int?
  featured Boolean @default(false)
  order Int @default(0)
  published Boolean @default(false)
  client Client @relation(fields: [clientId], references: [id])
  clientId String
}

model Lead {
  id String @id @default(cuid())
  name String
  email String
  phone String?
  company String?
  projectType String
  budget Int?
  message String
  status LeadStatus @default(NEW)
  createdAt DateTime @default(now())
}
```

Add `Service`, `BtsImage`, `Post`, `Faq`, `VendorApplication`, `SiteSettings` and `AdminUser` the same way (see PRD section 6.1).

3. Run `pnpm prisma migrate dev --name init`, then write `prisma/seed.ts` that loads your `content/*.json`, and run `pnpm prisma db seed`.
4. Replace JSON imports with Prisma queries in server components. Use `export const revalidate = 3600` and tag-based revalidation.
5. **Lead API** (`app/api/leads/route.ts`):

```
export async function POST(req: Request) {
  const body = await req.json();
  const ip = req.headers.get('x-forwarded-for') ?? 'anon';
  const { success } = await ratelimit.limit(ip);          // 5 per hour
  if (!success) return Response.json({ error: 'Too many requests' }, {
status: 429 });

  const data = leadSchema.parse(body);                    // Zod
  if (!(await verifyTurnstile(body.token))) return Response.json({ error:
'Bot check failed' }, { status: 400 });

  const lead = await db.lead.create({ data });            // save FIRST
  await Promise.allSettled([notifyOwner(lead), autoReply(lead)]);
  return Response.json({ ok: true });
}
```

6. Same pattern for `/api/vendors`. Build `lib/email.ts` with Resend templates.

Done when: submitting the form creates a DB row and sends both emails; spam attempts return 429/400.

7. Sprint 6: Admin CMS (Days 37–47)

1. Configure Auth.js (credentials or Google). Seed one admin. Protect `/admin` with `middleware.ts`.
2. Build a generic admin table + form component, then reuse it for Projects, Clients, Services, BTS, Posts, FAQs, Settings.
3. Upload route returns a Blob URL; form fields store that URL.
4. Reorder with drag-and-drop (`@dnd-kit/core`) that saves the `order` field.
5. After each save, call `revalidateTag()` so the public site updates within seconds.
6. Leads inbox: table, status dropdown, notes, CSV export.

Shortcut: replace steps 1–5 with Sanity Studio and fetch content through its API. This saves about 1.5 weeks.

Done when: a non-technical person can add a project in `/admin` and see it live on the site without a deploy.

8. Sprint 7: QA, Performance and Launch (Days 48–56)

Testing

- Unit (Vitest): Zod schemas, tier function, slug helpers.

- E2E (Playwright): home loads; open showreel; submit brief (mock email); filter portfolio; admin login and create project.
- Real devices: iPhone Safari, Android Chrome, one low-end phone.
- Network: throttle to Fast 3G and confirm hero poster shows quickly.
- Accessibility: keyboard-only pass, screen-reader spot check, reduced-motion check.

Performance checklist

- All images via `next/image` with correct `sizes`; hero poster `priority`.
- Videos never autoplay off-screen; pause with IntersectionObserver.
- Dynamic-import GSAP-heavy sections (`next/dynamic`).
- Bundle analyser: no section adds more than about 50 KB gzipped.
- Target: LCP under 2.5s, CLS under 0.1, mobile Lighthouse 85+.

9. Environment Variables

```
DATABASE_URL=
NEXTAUTH_URL=https://yourdomain.com
NEXTAUTH_SECRET=            # openssl rand -base64 32
RESEND_API_KEY=
NOTIFY_EMAIL=
BLOB_READ_WRITE_TOKEN=
TURNSTILE_SECRET_KEY=
NEXT_PUBLIC_TURNSTILE_SITE_KEY=
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=
SENTRY_DSN=
NEXT_PUBLIC_GA_ID=
REVALIDATE_SECRET=
```

10. Deployment Runbook

1. Merge `develop` → `main` through a pull request; CI must pass (lint, types, tests).
2. In Vercel → Project → Settings → Environment Variables, add every variable above for **Production** and **Preview** (use a separate preview database).
3. Set the build command to `prisma generate && prisma migrate deploy && next build`.
4. Deploy; open the production URL; run the smoke test: home, one case study, brief form, admin login.

5. Run the seed once against production: `pnpm prisma db seed` with the production `DATABASE_URL`.
6. **Domain:** Vercel → Domains → add `yourdomain.com` and `www`. At your registrar or Cloudflare, add the A record (apex) and CNAME (`www`) that Vercel displays. Set apex as primary and redirect `www`.
7. **Email:** in Resend add your domain; copy its SPF, DKIM and DMARC records into DNS; verify; send a test.
8. Submit `https://yourdomain.com/sitemap.xml` in Google Search Console and Bing Webmaster Tools.
9. Test link previews on WhatsApp, LinkedIn and X.
10. Turn on Sentry alerts, an uptime monitor for `/` and `/api/health`, and daily DB backups.

Rollback: Vercel → Deployments → pick the last good build → Promote to Production. Database changes: always make migrations backward-compatible.

11. Go-Live Checklist

- [ ] All routes load; no console errors; no broken links
- [ ] Brief form: DB row + owner email + auto-reply received
- [ ] Admin login works; edits appear on the live site
- [ ] Lighthouse mobile: Performance 85+, Accessibility 95+, SEO 95+
- [ ] Metadata, OG image, canonical, sitemap and robots correct
- [ ] Privacy policy and terms published; form consent text present
- [ ] Secrets only in Vercel; `.env` is gitignored
- [ ] Backups and monitoring active

12. Weekly Schedule at a Glance

Week	Focus	Output
1	Sprint 1–2	Deployed shell, nav, footer, UI kit
2–3	Sprint 3	Complete home page on mock data
4	Sprint 4	All inner pages + SEO
5	Sprint 5	Database, forms, emails
6	Sprint 6	Admin CMS

Week	Focus	Output
7–8	Sprint 7	QA, performance, launch

13. Your First Three Days (Start Here)

Day 1: install tools, create accounts, run `create-next-app`, push to GitHub, connect Vercel. **Day 2:** add design tokens and fonts; build Navbar, Footer and the mobile menu. **Day 3:** build the Hero with a poster fallback, then the Architecture section. Deploy and review the preview link on your phone.